CESI
ÉCOLE D'INGÉNIEURS

Stockage & Big Data

CER | Préparation des données

Mercredi 21 octobre 2026

Mazou Marine
Fisa Info 25–28
CESI

TABLE DES MATIÈRES

Étude de cas : Préparation des données **3**
AAV . . . 3
Mots clés . . . 3
Contexte . . . 3
Problématique . . . 4
Contraintes . . . 4
Généralisation . . . 4
Livrables . . . 4
Pistes de solutions . . . 4
Plan d'actions . . . 5

1 Définitions des mots clés **6**

2 Intégration des données : ETL ou ELT **8**
2.1 Le processus ETL . . . 8
2.2 Le processus ELT . . . 8
2.3 Tableau comparatif . . . 9
2.4 État de l'art du stockage et de la transformation . . . 10
2.5 Choix pour le CHU . . . 10

3 Qualification et préparation des données **11**
3.1 Qualifier un jeu de données . . . 11
3.2 Contrôles de qualité . . . 12
3.3 Conversion des dates . . . 13
3.4 Données structurées . . . 14
3.5 Données semi-structurées . . . 14
3.6 Données non structurées . . . 15

4 Confidentialité des données **16**
4.1 Mécanismes de protection . . . 16
4.2 Application aux données du CHU . . . 17

5 Organisation du Data Lake **18**
5.1 Zones du lac . . . 18
5.2 Arborescence et règles de nommage . . . 18
5.3 Gouvernance . . . 19

6 Plan de mise en place de l'alimentation **20**
6.1 Récupérer les données avec Talend . . . 20
6.2 Étapes . . . 20
6.3 Alimenter le modèle dimensionnel . . . 21
6.4 Chargement initial et incrémental . . . 22

7 Intégration dans une base NoSQL **23**
7.1 Quand utiliser une base NoSQL . . . 23

7.2 Modélisation et chargement 23

8 Stratégie d'analyse et requêtes **24**
8.1 Proposer une stratégie d'analyse 24
8.2 Traduire une demande en requêtes 25

Conclusion **27**

Capitalisation **28**

Ressources **29**

ÉTUDE DE CAS : PRÉPARATION DES DONNÉES

AAV

- 3 Alimenter un modèle dimensionnel (Data Mart)
- 3 Écrire des requêtes SQL utilisant plusieurs tables
- 3 Traduire une demande en requêtes
- 3 Utiliser un ETL pour récupérer des données
- 4 Qualifier un jeu de données
- 4 Proposer une stratégie d'analyse de données
- 3 Réaliser un traitement de données prenant en compte les contraintes RGPD (anonymisation)
- 3 Proposer une organisation type Data Lake
- 3 Utiliser un ETL pour intégrer des données dans une base NoSQL

MOTS CLÉS

- Les 3 V
- ETL / ELT
- Données structurées, semi-structurées et non structurées
- Charge de travail analytique

CONTEXTE

Une équipe doit alimenter l'entrepôt de données d'un système décisionnel. Les sources sont hétérogènes : structurées, semi-structurées et non structurées. Archie a prévu un stockage intermédiaire qui sert de source unique. Le modèle cible est déjà dessiné, avec plusieurs dimensions.

PROBLÉMATIQUE

Comment transformer des données hétérogènes pour alimenter l'entrepôt de données, et faut-il transformer les données avant ou après le chargement ?

CONTRAINTES

- Sources hétérogènes
- Confidentialité des données
- Modèle cible imposé
- Format de date imposé par Hive

GÉNÉRALISATION

- Transformation
- Chargement
- Confidentialité
- Préparation des données

LIVRABLES

- Analyse et comparaison des différentes méthodes (ETL, ELT. . .)
- Plan de mise en place de l'alimentation de l'entrepôt

PISTES DE SOLUTIONS

- Choisir entre ETL et ELT.
- Se renseigner sur l'état de l'art du stockage et de la transformation de données.
- Étudier les mécanismes qui garantissent la confidentialité.

PLAN D'ACTIONS

- Définir les mots clés, étudier les ressources et réaliser le workshop.
- Dresser un tableau comparatif des méthodes ETL et ELT.
- Étudier la conversion des données : dates, données structurées, semi-structurées et non structurées.
- Étudier les mécanismes de confidentialité et les appliquer aux données du CHU.
- Qualifier les jeux de données fournis et proposer une organisation du Data Lake.
- Établir le plan de mise en place de l'alimentation de l'entrepôt, y compris vers une base NoSQL.
- Proposer une stratégie d'analyse et traduire des demandes en requêtes SQL.

1 DÉFINITIONS DES MOTS CLÉS

Les 3 V: Définition du Big Data proposée par Doug Laney (Gartner) en 2001. Un ensemble de données relève du Big Data quand il se distingue par :

- **le volume** : une quantité de données qui dépasse les capacités d'un serveur unique. Le fichier des décès fourni compte à lui seul plus de 25 millions de lignes (1,9 Go) ;
- **la vélocité** : la vitesse à laquelle les données sont produites et doivent être traitées, du chargement nocturne au flux en temps réel ;
- **la variété** : la diversité des formats et des structures (tables, fichiers CSV, JSON, documents PDF, images).

La définition a ensuite été étendue à 5 V avec la véracité et la valeur (CER 1). Pour l'alimentation de l'entrepôt, c'est surtout la **variété** qui pose problème : chaque format demande sa propre conversion.

ETL / **ELT**: Deux façons d'enchaîner les trois opérations qui alimentent un entrepôt : **E**xtraire les données des sources, les **T**ransformer (nettoyage, conversion, calculs) et les **C**harger (*Load*) dans la cible.

- **ETL** : les données sont transformées **avant** le chargement, sur un serveur ou un outil d'intégration dédié (Talend, Informatica).
- **ELT** : les données sont chargées **brutes** dans la cible, puis transformées **sur place** avec la puissance de calcul de la cible (requêtes Hive ou Spark sur le cluster).

Les deux approches sont comparées dans la partie 2.

Données structurées, semi-structurées et non structurées: Les données se classent selon le degré d'organisation de leur contenu :

	Structurées	Semi-structurées	Non structurées
Schéma	Fixe, défini à l'avance	Souple, porté par les données (balises, clés)	Aucun
Formats	Tables SQL, CSV	JSON, XML, logs	Textes, PDF, images, sons
Interrogation	SQL directement	Après aplatissement ou avec un SerDe	Après extraction d'informations
Exemple CHU	Table `Patient`, fichier CSV des hospitalisations	Dossier patient échangé en JSON (norme FHIR)	Fiches descriptives PDF des indicateurs de satisfaction

Charge de travail analytique: Type de sollicitation d'une base de données propre au décisionnel. Elle se caractérise par :

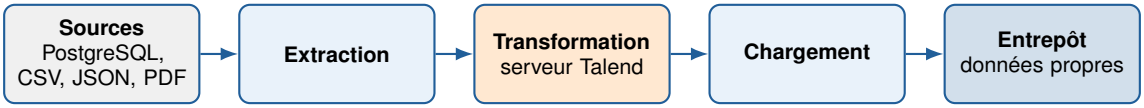
- des requêtes en **lecture** qui parcourent et agrègent de très nombreuses lignes (sommes, moyennes, comptages par période) ;
- des écritures rares et **en masse**, lors des chargements planifiés ;
- peu d'utilisateurs simultanés, mais des requêtes longues et imprévisibles.

Elle s'oppose à la charge **transactionnelle** (OLTP), faite de nombreuses petites lectures et écritures. Une charge analytique justifie un stockage orienté colonnes (ORC, Parquet), le partitionnement et le parallélisme du cluster.

2 INTÉGRATION DES DONNÉES : ETL OU ELT

2.1 LE PROCESSUS ETL

Dans un ETL, les données transitent par un moteur de transformation situé entre les sources et l'entrepôt. Seules des données propres et conformes au modèle cible sont chargées.



+ La cible ne reçoit que des données validées et conformes.
+ Les données sensibles peuvent être pseudonymisées **avant** d'arriver dans le cluster.
+ Les règles de transformation sont centralisées et documentées dans l'outil (jobs Talend).
– Le serveur de transformation devient un goulot d'étranglement sur de gros volumes.
– Les données brutes ne sont pas conservées : un nouveau besoin oblige à réextraire les sources.

2.2 LE PROCESSUS ELT

Dans un ELT, les données sont d'abord copiées brutes dans la cible (Data Lake ou entrepôt), puis transformées par des requêtes exécutées sur la cible elle-même.



+ La transformation profite de la puissance du cluster et passe à l'échelle avec lui.
+ Les données brutes sont conservées : on peut rejouer ou modifier une transformation sans réextraire.
+ Chargement rapide, la transformation est faite plus tard, à la demande.
– Des données brutes, donc potentiellement identifiantes, sont stockées dans le cluster.
– La qualité des données doit être gérée dans la cible, avec des règles dispersées dans des scripts SQL.

2.3 TABLEAU COMPARATIF

Critère	ETL	ELT
Ordre	Extraire, transformer, charger	Extraire, charger, transformer
Lieu de la transformation	Serveur ou outil d'intégration	Cible (entrepôt, cluster)
Données chargées	Nettoyées, conformes au modèle	Brutes, tous formats
Volumes	Moyens, limités par le serveur ETL	Très gros, limités par le cluster
Temps de chargement	Plus long (transformation au passage)	Court (copie directe)
Données brutes	Non conservées	Conservées, transformations rejouables
Confidentialité	Données masquées avant la cible	Données sensibles présentes dans la cible
Qualité	Contrôlée avant le chargement	Contrôlée après, dans la cible
Outils	Talend, Informatica, SSIS	Hive, Spark, dbt, entrepôts cloud
Cas d'usage	Entrepôt classique, données sensibles	Data Lake, Big Data, entrepôts cloud

2.4 ÉTAT DE L'ART DU STOCKAGE ET DE LA TRANSFORMATION

Les architectures ont évolué avec les volumes et le cloud :

- **Entrepôt classique** (années 1990) : ETL nocturne vers un SGBD relationnel. Modèle stable, mais rigide et coûteux à faire grandir.
- **Data Lake** (années 2010) : stockage brut sur HDFS ou stockage objet (S3), schéma à la lecture. L'ELT devient naturel, au risque d'un « marécage » de données non documentées.
- **Entrepôts cloud** (Snowflake, BigQuery, Redshift) : stockage et calcul séparés et élastiques. L'ELT y est la norme, avec des transformations écrites en SQL et versionnées (dbt).
- **Lakehouse** (Delta Lake, Apache Iceberg) : des tables transactionnelles directement sur le Data Lake, qui réunissent souplesse du lac et fiabilité de l'entrepôt.
- **Flux continus** : la capture des changements (*Change Data Capture*) et des outils comme Kafka alimentent la cible au fil de l'eau plutôt qu'en lot nocturne.

Dans la pratique, les deux approches se combinent souvent : une transformation légère avant le chargement, puis les transformations lourdes dans la cible. On parle d'**EtLT**.

2.5 CHOIX POUR LE CHU

Pour le CHU, ni l'ETL pur ni l'ELT pur ne convient :

- un **ELT pur** déposerait dans HDFS des noms, des numéros de sécurité sociale et des diagnostics en clair, ce que le RGPD et l'architecture du CER 1 excluent ;
- un **ETL pur** ferait porter à Talend des traitements lourds (25 millions de décès, jointures entre sources) que le cluster sait mieux paralléliser.

On retient donc un **EtLT**, cohérent avec l'architecture proposée au CER 1 :

Étape	Outil	Traitements
E + t (léger)	Talend	Extraction, encodage en UTF-8, formats de date, pseudonymisation des identifiants, rejet des lignes invalides
L	Talend	Dépôt dans HDFS, zone brute (données déjà pseudonymisées)
T (lourd)	Hive	Jointures, calcul des mesures, alimentation des dimensions et des faits du modèle cible, format ORC partitionné

Les données sensibles ne sont jamais écrites en clair dans le cluster, et les transformations volumineuses profitent du parallélisme de Hive.

3 QUALIFICATION ET PRÉPARATION DES DONNÉES

3.1 QUALIFIER UN JEU DE DONNÉES

Qualifier un jeu de données, c'est mesurer ce qu'il contient avant de l'utiliser : volume, structure, complétude, unicité, plages de valeurs et liens avec les autres sources. Ce profilage (*data profiling*) dit si les données peuvent répondre au besoin, et quelles transformations seront nécessaires. Il a été réalisé sur trois fichiers fournis :

Critère	Hospitalisations	Établissements	Professionnels
Volume	2 479 lignes, 7 colonnes	416 665 lignes, 24 colonnes	1 048 575 lignes, 9 colonnes
Format	CSV ;, ASCII	CSV ;, UTF-8	CSV ;, ISO-8859-1, CRLF
Complétude	100 % sur toutes les colonnes	100 % sur l'identifiant et la raison sociale, 67 % sur la commune, 14 % sur l'e-mail, FINESS du site quasi vide	100 % sur l'identité et la profession, 64 % sur la commune, 18 % sur la spécialité
Unicité	`Num_Hospitalisation` unique, 2 465 patients distincts	`identifiant_organisation` unique	`identifiant` unique
Plages	Entrées du 21/06/2015 au 19/06/2021, durées de 1 à 30 jours (moyenne 15,4)	–	–
Liens	100 % des établissements retrouvés dans le fichier des établissements	Clé `identifiant_organisation`, et non le FINESS	–

Ce profilage fait ressortir trois points qui guident la préparation :

- le fichier des professionnels compte exactement 1 048 576 lignes avec l'en-tête, soit la limite d'une feuille Excel : il a très probablement été **tronqué** lors d'un passage par Excel. Il faut récupérer la source complète avant de l'utiliser comme référentiel ;
- les hospitalisations référencent les établissements par `identifiant_organisation`, alors que les fichiers de satisfaction utilisent le **FINESS**, presque jamais renseigné dans le fichier des établissements. Le rapprochement satisfaction–établissement demande donc une table de correspondance ;
- le fichier des hospitalisations est complet et cohérent, mais petit (2 479 séjours) : il convient pour la maquette, pas pour des statistiques fines par établissement.

Le résultat de la qualification est consigné dans une fiche par source (volume, format, taux de complétude par colonne, anomalies, décision), mise à jour à chaque nouvelle livraison de données.

3.2 CONTRÔLES DE QUALITÉ

Avant toute conversion, chaque source est contrôlée. Les lignes qui échouent ne sont pas chargées mais écrites dans un fichier de rejets, pour être corrigées ou signalées :

Contrôle	Vérification	Exemple CHU
Complétude	Les champs obligatoires sont renseignés	Une hospitalisation sans date d'entrée est rejetée
Unicité	Pas de doublon sur la clé	Un même `Num_Hospitalisation` présent deux fois
Format	Le type et le format sont respectés	Une date `31/02/2017` ou un âge négatif
Cohérence	Les valeurs sont plausibles entre elles	Une date de décès antérieure à la date de naissance
Intégrité	Les références existent	Un `identifiant_organisation` absent du fichier des établissements

Dans Talend, ces contrôles se font avec les composants `tSchemaComplianceCheck` (types et formats), `tUniqRow` (doublons) et `tMap` (règles et recherches dans les référentiels), dont la sortie de rejet alimente un fichier d'erreurs.

3.3 CONVERSION DES DATES

Hive impose ses formats : le type `DATE` attend `yyyy-MM-dd` et le type `TIMESTAMP` attend `yyyy-MM-dd HH:mm:ss`. Une date dans un autre format n'est pas convertie : Hive renvoie `NULL`, sans erreur. Or les sources fournies n'utilisent pas toutes le même format :

Source	Format d'origine	Format Hive
`Hospitalisations.csv` (`Date_Entree`)	`27/09/2017`	`2017-09-27`
`deces.csv` (`date_naissance`, `date_deces`)	`1903-11-11`	déjà conforme
Table `Consultation` (`Date`, `Heure_debut`)	type `date` et `time`	`TIMESTAMP` recomposé
Table `Patient` (`Date`)	texte de 10 caractères	à analyser puis convertir

La conversion peut se faire à deux endroits :

Dans Talend (avant chargement)

```
TalendDate.formatDate("yyyy-MM-dd",
  TalendDate.parseDate("dd/MM/yyyy",
    row1.Date_Entree))
```

Dans Hive (après chargement)

```
TO_DATE(FROM_UNIXTIME(
  UNIX_TIMESTAMP(date_entree,
    'dd/MM/yyyy')))
```

On convertit dans Talend, au moment de l'extraction : la date est contrôlée en même temps, et une date invalide part dans les rejets au lieu de devenir un `NULL` silencieux dans Hive. Une **dimension Temps** est ensuite générée une fois pour toutes (une ligne par jour, avec jour, mois, trimestre, année), et les faits la référencent par une clé `AAAAMMJJ`.

3.4 DONNÉES STRUCTURÉES

Les données structurées ont déjà un schéma, mais celui-ci varie d'une source à l'autre. Les fichiers fournis montrent les écarts à harmoniser :

- **encodage** : `etablissement_sante.csv` est en UTF-8, `professionnel_sante.csv` en ISO-8859-1 et les résultats de satisfaction 2019 en Windows-1252. Lus sans précaution, les accents deviennent illisibles (« Auvergne-Rh**?**ne-Alpes »). Tout est converti en UTF-8, l'encodage attendu par Hive ;
- **séparateur** : point-virgule pour les fichiers d'établissements et d'hospitalisations, virgule pour les décès ;
- **fins de ligne** : Windows (CRLF) pour certains fichiers, Unix (LF) pour d'autres ;
- **types** : identifiants à zéros initiaux (`010001717`) à garder en texte, nombres décimaux à virgule à convertir avec un point ;
- **doublons de format** : un fichier de satisfaction 2016 existe à la fois en CSV et en Excel, on n'en charge qu'un.

Les tables PostgreSQL sont lues avec `tPostgresqlInput`, les CSV avec `tFileInputDelimited`, en précisant pour chacun l'encodage, le séparateur et la ligne d'en-tête.

3.5 DONNÉES SEMI-STRUCTURÉES

Les données semi-structurées (JSON, XML) sont imbriquées : un document peut contenir des listes et des sous-objets, alors qu'une table est plate. Les données de santé sont souvent échangées ainsi, par exemple au format JSON de la norme **FHIR** :

Document JSON

```
{ "id_patient":  29620,
  "sejours":  [
    {"entree":  "2017-09-27",
     "diag":  "S02800"},
    {"entree":  "2020-08-16",
     "diag":  "Q902"} ] }
```

Après aplatissement

id_patient	entree	diag
29620	2017-09-27	S02800
29620	2020-08-16	Q902

Chaque élément de la liste devient une ligne, et la clé du document est répétée. Dans Talend, `tExtractJSONFields` lit le document avec des chemins JSONPath (`$.sejours[*]`). Dans Hive, on peut aussi charger le JSON brut avec un *SerDe* JSON, puis l'aplatir avec `LATERAL VIEW explode(sejours)`.

3.6 DONNÉES NON STRUCTURÉES

Les données non structurées (textes libres, PDF, images) ne peuvent pas être chargées telles quelles dans un modèle dimensionnel. On les traite en deux temps :

1. le fichier est conservé **brut dans le Data Lake**, avec un identifiant et un chemin ;
2. on en **extrait des informations structurées**, qui seules rejoignent l'entrepôt : métadonnées (date, auteur, type), mots clés, codes reconnus dans le texte (codes CIM-10 d'un compte rendu), ou résultat d'un modèle d'analyse (classification d'une image).

Dans les données fournies, les fiches descriptives des indicateurs de satisfaction sont des PDF : on en retient les définitions des indicateurs pour documenter la dimension correspondante, et le PDF reste consultable dans le lac. Un texte libre peut aussi contenir des informations identifiantes (nom d'un patient dans un compte rendu) : il doit être pseudonymisé avant toute exploitation.

4 CONFIDENTIALITÉ DES DONNÉES

4.1 MÉCANISMES DE PROTECTION

La confidentialité repose sur deux familles de mécanismes : ceux qui **transforment la donnée** pour qu'elle ne permette plus d'identifier une personne, et ceux qui **protègent l'accès** à la donnée.

Technique	Principe	Réversible	Exemple CHU
Suppression	Retirer la colonne	Non	Nom, prénom, e-mail, téléphone
Masquage	Cacher une partie de la valeur	Non	`06 ** ** ** 12`
Hachage salé	Remplacer par une empreinte (SHA-256 + clé secrète)	Non	`Id_patient` → `a3f9...`
Tokenisation	Remplacer par un jeton, avec une table de correspondance protégée	Oui	Numéro de sécurité sociale
Généralisation	Rendre la valeur moins précise	Non	Date de naissance → tranche d'âge, adresse → département
Agrégation	Ne publier que des totaux	Non	Nombre de séjours par service et par mois
Bruit	Perturber légèrement les valeurs (confidentialité différentielle)	Non	Effectifs arrondis dans les tableaux publiés
Chiffrement	Rendre la donnée illisible sans clé	Oui	Disques et échanges réseau du cluster
Contrôle d'accès	Limiter qui voit quoi	–	Zone brute réservée aux data engineers

Le RGPD distingue deux résultats :

- la **pseudonymisation** remplace les identifiants, mais la personne reste réidentifiable avec une information complémentaire (la clé, la table de correspondance). Les données restent personnelles et soumises au RGPD ;
- l'**anonymisation** rend la réidentification impossible, de façon irréversible. Les données sortent alors du champ du RGPD. Selon le G29 (devenu le CEPD), elle doit empêcher l'individualisation, la corrélation et l'inférence.

4.2 APPLICATION AUX DONNÉES DU CHU

La table `Patient` concentre les données identifiantes. Chaque colonne reçoit un traitement, appliqué dans Talend avant le dépôt dans HDFS :

Colonne	Traitement	Résultat dans le cluster
`Id_patient`	Hachage salé (SHA-256)	Clé pseudonymisée, identique dans toutes les sources pour permettre les jointures
`Nom, Prenom`	Suppression	–
`EMail, Tel`	Suppression	–
`Num_Secu`	Suppression	–
`Adresse, Ville`	Suppression	–
`Code_postal`	Généralisation	Département
`Date, Age`	Généralisation	Tranche d'âge
`Sexe`	Conservé	Utile à l'analyse
`Groupe_sanguin, Poid, Taille`	Minimisation	Chargés seulement si un indicateur les utilise

Le hachage utilise un **sel secret** : sans lui, un identifiant numérique se retrouverait en hachant toutes les valeurs possibles. Ce sel est conservé hors du cluster, et la même valeur est utilisée pour toutes les sources, ce qui garde les jointures possibles entre consultations, prescriptions et hospitalisations.

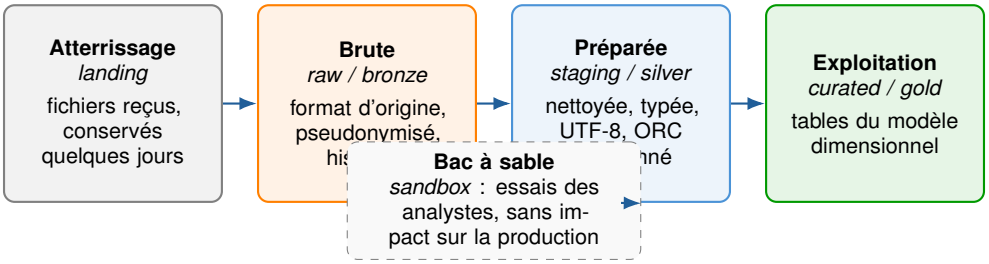
Le fichier des décès contient lui aussi des noms, prénoms et dates de naissance. Il ne sert qu'à des statistiques de mortalité : on ne garde que le sexe, l'âge au décès, la date et la commune du décès. Les noms ne sont jamais chargés.

Enfin, les indicateurs publiés dans Power BI sont **agrégés**, et les cases portant sur de très petits effectifs (moins de 10 patients par exemple) sont masquées pour éviter qu'un patient soit reconnaissable.

5 ORGANISATION DU DATA LAKE

5.1 ZONES DU LAC

Un Data Lake sans organisation devient vite un « marécage » (*data swamp*) : des fichiers dont on ne connaît plus l'origine, la qualité ni le propriétaire. On le découpe donc en zones, que les données traversent de gauche à droite en gagnant en qualité :



Zone	Contenu	Accès	Conservation
Atterrissage	Fichiers tels que livrés	Jobs d'ingestion seulement	Quelques jours
Brute	Copie fidèle, pseudonymisée, jamais modifiée	Data engineers	Durée légale définie
Préparée	Données contrôlées et converties	Data engineers, data scientists	Selon les besoins
Exploitation	Faits et dimensions	Analystes, Power BI (lecture)	Historique complet
Bac à sable	Jeux de test, analyses exploratoires	Analystes	Purge régulière

5.2 ARBORESCENCE ET RÈGLES DE NOMMAGE

Chaque zone suit la même arborescence dans HDFS : zone, domaine, source, puis date de chargement. On retrouve ainsi l'origine de chaque fichier, et une partition par date permet de rejouer ou de supprimer un chargement précis.

Arborescence HDFS du lac CHU

```
/data/chu/
  landing/postgresql/consultation/2026-10-21/
  raw/soins/hospitalisations/date_chargement=2026-10-21/
  raw/referentiel/etablissements/date_chargement=2026-10-21/
  raw/open_data/deces/date_chargement=2026-10-21/
  staging/soins/hospitalisations/annee=2021/part-00000.orc
  curated/dwh/fait_hospitalisation/annee=2021/
  sandbox/<analyste>/
```

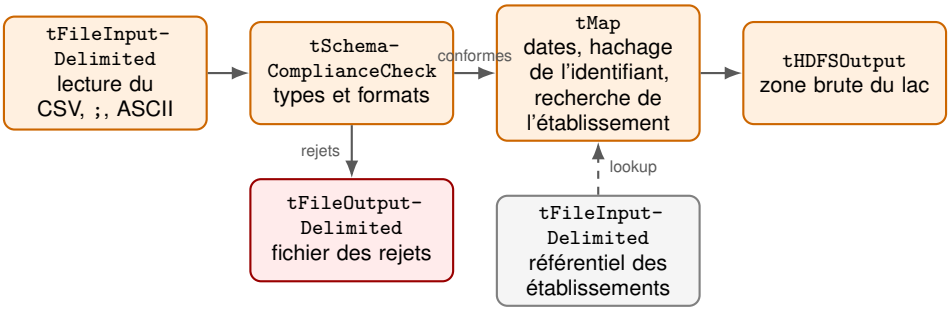
5.3 GOUVERNANCE

- **Catalogue de métadonnées** : chaque jeu de données est décrit (source, propriétaire, date, schéma, fiche de qualification). Dans Cloudera, le métastore Hive et Navigator ou Atlas jouent ce rôle.
- **Lignage** : on sait de quel fichier brut provient chaque table de l'entrepôt, et par quel job.
- **Droits par zone** : les zones brute et préparée sont réservées aux équipes techniques, l'entrepôt est ouvert en lecture aux analystes (Sentry).
- **Cycle de vie** : chaque zone a une durée de conservation, appliquée en supprimant les partitions de date expirées.

6 PLAN DE MISE EN PLACE DE L'ALIMENTATION

6.1 RÉCUPÉRER LES DONNÉES AVEC TALEND

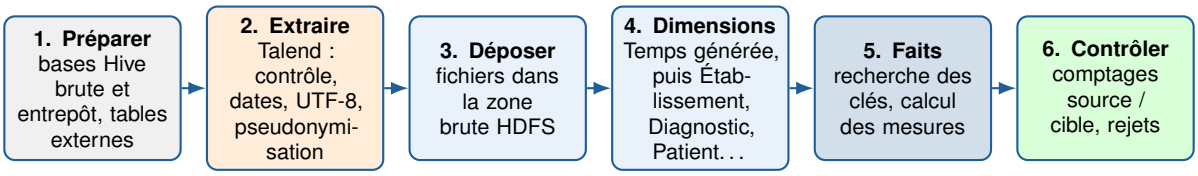
Dans Talend Open Studio for Big Data, chaque source est récupérée par un **job**, assemblage graphique de composants reliés par des flux de lignes. Le job des hospitalisations enchaîne ainsi :



Les tables PostgreSQL sont récupérées de la même façon avec `tPostgresqlInput` et une requête SQL qui ne sélectionne que les colonnes utiles (minimisation). Les paramètres (chemins, connexion, sel de hachage) sont placés dans des **variables de contexte**, différentes entre la maquette et la production, et jamais écrites en dur dans le job.

6.2 ÉTAPES

L'alimentation suit un ordre imposé par les clés : les dimensions sont chargées avant les faits, car chaque ligne de faits doit retrouver la clé de ses dimensions.



1. **Préparer la cible** : créer dans Hive une base pour la zone brute (tables externes pointant sur les dossiers HDFS) et une base pour l'entrepôt (tables du modèle cible au format ORC).
2. **Extraire et préparer** avec Talend : lecture de chaque source avec son encodage et son séparateur, contrôles de qualité, conversion des dates, pseudonymisation des identifiants, écriture des rejets.
3. **Déposer** les fichiers obtenus dans la zone brute de HDFS (`tHDFSOutput`), un dossier par source et par date de chargement.
4. **Charger les dimensions** : la dimension Temps est générée par un script, les autres sont alimentées depuis la zone brute par des requêtes Hive (`INSERT OVERWRITE ... SELECT DISTINCT`), avec une clé de substitution.
5. **Charger les faits** : chaque ligne retrouve la clé de ses dimensions par jointure, les mesures sont calculées (durée de séjour, durée de consultation), puis les données sont insérées dans la partition de leur année.
6. **Contrôler** : comparer les nombres de lignes entre source, rejets et cible, et vérifier qu'aucun fait n'a de clé de dimension vide.

6.3 ALIMENTER LE MODÈLE DIMENSIONNEL

Le modèle cible est la constellation définie au CER 1. Une fois les données préparées, le chargement se fait en SQL dans Hive : d'abord les dimensions, puis les faits qui retrouvent leurs clés par jointure.

Dimension Établissement : une ligne par établissement

```
INSERT OVERWRITE TABLE dwh.dim_etablissement
SELECT e.identifiant_organisation, e.raison_sociale_site,
       e.commune, SUBSTR(e.code_postal, 1, 2) AS departement
FROM staging.etablissement e;
```

Fait Hospitalisation : clés des dimensions et mesures

```
SET hive.exec.dynamic.partition.mode=nonstrict;
INSERT OVERWRITE TABLE dwh.fait_hospitalisation PARTITION (annee)
SELECT h.num_hospitalisation,
       h.id_patient_hash,    -- déjà pseudonymisé par Talend
       e.identifiant_organisation,
       d.code_diag,
       CAST(REGEXP_REPLACE(CAST(h.date_entree AS STRING), '-', '') AS INT)
             AS id_date_entree,
       h.jour_hospitalisation AS duree_sejour,
       1 AS nb_hospitalisations,
       YEAR(h.date_entree) AS annee
FROM staging.hospitalisation h
JOIN dwh.dim_etablissement e ON e.identifiant_organisation = h.identifiant_organisation
JOIN dwh.dim_diagnostic d    ON d.code_diag = h.code_diagnostic;
```

Les jointures internes écartent les faits dont une dimension est introuvable : on les compte avant et après chargement, et l'écart doit correspondre aux rejets attendus. Une variante consiste à rattacher ces lignes à une ligne « Inconnu » de la dimension, pour ne perdre aucun fait.

6.4 CHARGEMENT INITIAL ET INCRÉMENTAL

	Chargement initial	Chargement incrémental
Quand	Une fois, à la mise en service	Chaque nuit, planifié
Données	Tout l'historique des sources	Lignes nouvelles ou modifiées depuis le dernier passage
Détection	–	Date de modification, identifiant croissant (`Num_consultation`) ou capture des changements
Écriture Hive	`INSERT OVERWRITE` de toutes les partitions	Ajout dans la partition de l'année, ou réécriture de la seule partition concernée
Dimensions	Création complète	Ajout des nouvelles valeurs, historisation des changements (SCD type 2)

Les jobs Talend sont enchaînés par un job principal et planifiés la nuit, hors des heures d'utilisation de Power BI. Chaque exécution est journalisée (`tLogCatcher`, `tStatCatcher`) : date, nombre de lignes lues, chargées et rejetées. Un échec arrête la chaîne avant le chargement des faits, pour ne jamais laisser l'entrepôt dans un état partiel.

7 INTÉGRATION DANS UNE BASE NOSQL

7.1 QUAND UTILISER UNE BASE NOSQL

L'entrepôt Hive sert les agrégations. Pour lire très vite les données d'**un seul** élément, comme le parcours d'un patient, une base NoSQL est plus adaptée : on la modélise selon la requête à servir.

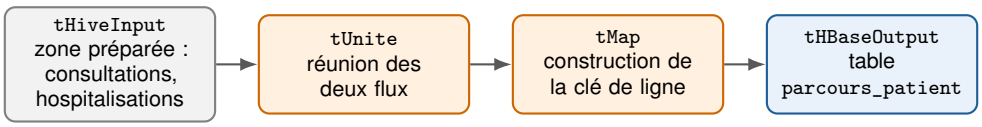
	HBase (dans Cloudera)	MongoDB
Modèle	Orienté colonnes : clé de ligne, familles de colonnes	Documents JSON
Stockage	Sur HDFS, dans le cluster existant	Serveur dédié
Accès type	Lecture par clé ou par plage de clés	Requêtes sur les champs du document
Composant Talend	`tHBaseConnection`, `tHBaseOutput`	`tMongoDBConnection`, `tMongoDBOutput`

HBase est retenu : fourni par Cloudera, il stocke ses données dans HDFS, sous les mêmes règles de sécurité que le lac.

7.2 MODÉLISATION ET CHARGEMENT

La table `parcours_patient` sert le **parcours de soins** : toutes les consultations et hospitalisations d'un patient, de la plus récente à la plus ancienne :

- **clé de ligne** : `<id_patient_hash>#<date inversée>`. Toutes les lignes d'un patient sont contiguës et triées de la plus récente à la plus ancienne ;
- **famille** `evt` : type d'événement (consultation, hospitalisation), date, code diagnostic ;
- **famille** `lieu` : établissement et professionnel.



Lire un parcours revient à balayer les clés de préfixe `<id_patient_hash>#`, en quelques millisecondes quel que soit le volume. HBase ne reçoit que des identifiants pseudonymisés.

8 STRATÉGIE D'ANALYSE ET REQUÊTES

8.1 PROPOSER UNE STRATÉGIE D'ANALYSE

Une stratégie d'analyse part des questions des décideurs, pas des données disponibles. Elle se construit en cinq étapes :

1. **recueillir les questions** métier auprès des utilisateurs (direction, chefs de service) ;
2. les **traduire en indicateurs** (KPI) avec leur définition exacte, leur axe d'analyse et leur fréquence de mise à jour ;
3. **vérifier les données** : chaque indicateur doit être calculable à partir d'une source qualifiée ;
4. choisir le **type d'analyse** : descriptive (que s'est-il passé ?), diagnostique (pourquoi ?), prédictive (que va-t-il se passer ?) ou prescriptive (que faire ?) ;
5. choisir la **restitution** : tableau de bord Power BI, rapport périodique ou requête ponctuelle.

Question métier	Indicateur	Type	Source
Quels établissements accueillent le plus de séjours ?	Nombre d'hospitalisations par établissement et par an	Descriptive	Fait Hospitalisation
Les séjours s'allongent-ils ?	Durée moyenne de séjour par année	Descriptive	Fait Hospitalisation
Quels diagnostics mènent aux séjours les plus longs ?	Durée moyenne par diagnostic	Diagnostique	Faits Hospitalisation et Diagnostic
La satisfaction suit-elle la charge ?	Score de satisfaction et nombre de séjours par établissement	Diagnostique	Faits Satisfaction et Hospitalisation
Combien de séjours l'an prochain ?	Prévision mensuelle du nombre de séjours	Prédictive	Historique des séjours

Pour la maquette, on commence par les analyses **descriptives** : elles sont directement calculables dans l'entrepôt et valident le chargement. Les analyses prédictives viendront ensuite, une fois l'historique suffisant.

8.2 TRADUIRE UNE DEMANDE EN REQUÊTES

Pour passer d'une demande exprimée en français à une requête SQL, on identifie successivement :

- la **mesure** demandée et son agrégation (compter, sommer, faire une moyenne) → `SELECT` et fonction d'agrégation ;
- les **axes** (« par établissement », « par année ») → `GROUP BY` ;
- les **filtres** (« en 2020 », « en Bretagne ») → `WHERE`, ou `HAVING` s'ils portent sur un résultat agrégé ;
- les **tables** qui portent ces informations → la table de faits et les dimensions à joindre ;
- l'**ordre** et la **limite** (« les 10 premiers ») → `ORDER BY` et `LIMIT`.

Demande 1 : « Quels sont les 10 établissements qui ont accueilli le plus de séjours en 2020 ? »

```
SELECT e.raison_sociale, COUNT(*) AS nb_sejours
FROM dwh.fait_hospitalisation f
JOIN dwh.dim_etablissement e
    ON e.identifiant_organisation = f.identifiant_organisation
WHERE f.annee = 2020            -- filtre sur la partition :  seule 2020 est lue
GROUP BY e.raison_sociale
ORDER BY nb_sejours DESC
LIMIT 10;
```

Demande 2 : « Quelle est la durée moyenne de séjour par chapitre de diagnostic et par année, pour les chapitres d'au moins 50 séjours ? »

```
SELECT d.chapitre, t.annee,
       ROUND(AVG(f.duree_sejour), 1) AS duree_moyenne,
       COUNT(*) AS nb_sejours
FROM dwh.fait_hospitalisation f
JOIN dwh.dim_diagnostic d ON d.code_diag = f.code_diag
JOIN dwh.dim_temps  t ON t.id_date = f.id_date_entree
GROUP BY d.chapitre, t.annee
HAVING COUNT(*) >= 50           -- filtre sur un résultat agrégé
ORDER BY t.annee, duree_moyenne DESC;
```

Demande 3 : « Les établissements qui accueillent le plus de séjours ont-ils une moins bonne satisfaction ? »

```
SELECT e.raison_sociale, s.annee, s.score_global,
       COUNT(f.num_hospitalisation) AS nb_sejours
FROM dwh.fait_satisfaction s
JOIN dwh.dim_etablissement e
    ON e.identifiant_organisation = s.identifiant_organisation
LEFT JOIN dwh.fait_hospitalisation f
    ON f.identifiant_organisation = s.identifiant_organisation
    AND f.annee = s.annee
GROUP BY e.raison_sociale, s.annee, s.score_global
ORDER BY nb_sejours DESC;
```

Cette troisième requête croise deux tables de faits par leurs dimensions communes (établissement et année) : c'est l'intérêt de la constellation. La jointure externe (`LEFT JOIN`) garde les établissements évalués même sans séjour enregistré. Elle suppose la table de correspondance FINESS–`identifiant_organisation` relevée lors de la qualification.

CONCLUSION

En réponse à la problématique, transformer des données hétérogènes pour l'entrepôt du CHU demande de traiter chaque type à part : harmoniser l'encodage, le séparateur et les types des données structurées, aplatir les données semi-structurées, et n'extraire des données non structurées que des informations exploitables. Les dates sont toutes ramenées au format imposé par Hive (`yyyy-MM-dd`) dès l'extraction, pour qu'une date invalide soit rejetée plutôt que transformée en valeur vide.

Faut-il transformer avant ou après le chargement ? Ni l'un ni l'autre exclusivement. L'ELT exploite la puissance du cluster et conserve les données brutes, mais il déposerait des données identifiantes dans HDFS. L'ETL protège ces données, mais devient un goulot sur les gros volumes. On retient un **EtLT** : Talend applique avant le chargement les transformations légères et celles qu'impose la confidentialité (pseudonymisation, suppression des identités), puis Hive réalise les transformations lourdes dans le cluster.

Avant toute transformation, chaque source est **qualifiée** : le profilage des fichiers fournis a révélé un fichier de professionnels tronqué à la limite d'Excel et une clé FINESS presque absente du référentiel des établissements, deux points à régler avant de charger. Les données traversent ensuite un Data Lake organisé en zones (atterrissage, brute, préparée, exploitation), avec une arborescence et des droits propres à chaque zone.

Le plan d'alimentation charge d'abord les dimensions puis les faits, par des jobs Talend et des requêtes Hive, avec un contrôle à chaque étape, en chargement initial puis incrémental nocturne. Une base NoSQL HBase complète l'entrepôt pour un usage précis, la consultation rapide du parcours d'un patient. Enfin, la stratégie d'analyse part des questions métier pour définir les indicateurs, et chaque demande se traduit en requête SQL sur la constellation.

CAPITALISATION

AAV	Commentaire	Statut
Alimenter un modèle dimensionnel (Data Mart)	Ordre de chargement (dimensions puis faits), requêtes Hive d'alimentation d'une dimension et d'un fait partitionné, contrôle des lignes écartées.	Acquis
Écrire des requêtes SQL utilisant plusieurs tables	Requêtes avec jointures internes et externes entre faits et dimensions, dont une qui croise deux tables de faits de la constellation.	Acquis
Traduire une demande en requêtes	Méthode (mesure, axes, filtres, tables, ordre) appliquée à trois demandes métier, avec `GROUP BY`, `HAVING` et `LIMIT`.	Acquis
Utiliser un ETL pour récupérer des données	Job Talend détaillé (lecture, contrôle, `tMap`, rejets, dépôt HDFS), variables de contexte, gestion des encodages et des séparateurs des sources.	Acquis
Qualifier un jeu de données	Profilage de trois fichiers réels (volume, format, complétude, unicité, plages, liens), avec deux anomalies relevées : fichier tronqué et clé FINESS absente.	Acquis
Proposer une stratégie d'analyse de données	Démarche en cinq étapes, des questions métier aux indicateurs, types d'analyse et restitution, appliquée au CHU.	Acquis
Réaliser un traitement de données prenant en compte les contraintes RGPD (anonymisation)	Techniques comparées, pseudonymisation et anonymisation distinguées, traitement colonne par colonne de la table Patient et du fichier des décès.	Acquis
Proposer une organisation type Data Lake	Zones (atterrissage, brute, préparée, exploitation, bac à sable), arborescence HDFS, règles de nommage et gouvernance.	Acquis
Utiliser un ETL pour intégrer des données dans une base NoSQL	Choix de HBase, conception de la clé de ligne pour le parcours patient et job Talend d'alimentation (`tHBaseOutput`). Non testé sur la machine Cloudera.	Partiel

RESSOURCES

- **Documentation Talend** : https://help.talend.com/
- **Apache Hive, types de données** : https://cwiki.apache.org/confluence/display/Hive/LanguageManual+Types
- **Apache Hive, fonctions de date** : https://cwiki.apache.org/confluence/display/Hive/LanguageManual+UDF
- **CNIL, l'anonymisation des données personnelles** : https://www.cnil.fr/fr/technologies/lanonymisation-de-donnees-personnelles
- **HL7 FHIR (échange de données de santé)** : https://hl7.org/fhir/
- **Kimball Group, techniques ETL** : https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/etl-architecture-34-subsystems/
- **Apache HBase, guide de référence** : https://hbase.apache.org/book.html
- **Apache Hive, jointures** : https://cwiki.apache.org/confluence/display/Hive/LanguageManual+Joins